



Universidad de Valladolid

Escuela Técnica Superior de Ingenieros de
Telecomunicación

Configuración y actualización dinámica de modelos
de aprendizaje profundo

GRADO EN INGENIERÍA DE TECNOLOGÍAS DE LA TELECOMUNICACIÓN

TRABAJO DE FIN DE GRADO

Autor:
Andrés García Treviño

Tutor:
Dr. D. Pablo Casaseca de la Higuera

Valladolid, junio 2026

Resumen

El crecimiento exponencial del número de dispositivos capaces de generar tráfico de red ha aumentado significativamente la necesidad de organizar los recursos disponibles para asegurar un nivel estable de calidad de servicio. La predicción del tráfico de red se ha convertido en la mayor aliada para evitar congestiones y pérdidas de servicio. Sin embargo, los modelos predictivos tradicionales tienen muchas dificultades para seguir el ritmo de cambio del tráfico de red actual. Como el tráfico de red puede ser modelado como una secuencia temporal, en este trabajo se estudian y comparan varios modelos de redes neuronales de aprendizaje profundo aplicadas a la predicción de series temporales con capacidad de adaptarse a las variaciones y anomalías mediante entrenamiento incremental. Se han estudiado varias arquitecturas que combinan redes convolucionales con redes recurrentes LSTM sobre el dataset Milan, que recoge el tráfico de red de la ciudad de Milán, Italia, entre noviembre y diciembre de 2013.

Los resultados de los modelos, analizados a través de diferentes métricas estadísticas, demuestran que los modelos que integran capas convolucionales y *online learning* son ampliamente superiores a los modelos tradicionales, incluso en situaciones de cambios bruscos en el patrón de los datos. El modelo ideal, presentando una capa CNN y una capa LSTM llega a reducir el error (NRMSE) cometido en las predicciones desde un 0.1778 conseguido con un modelo neuronal LSTM sencillo hasta un 0.142. Por último, los modelos analizados en este trabajo han sido optimizados al máximo para conseguir el mayor rendimiento posible en cuanto a carga computacional y tiempo de ejecución.

Palabras clave

Online learning, predicción, redes neuronales, redes convolucionales, tráfico de red.

Abstract

The exponential growth of telecommunication devices has raised the need of correctly allocating the network resources to ensure a reliable Quality of Service level. Network traffic prediction has become the greatest ally in the battle against network congestion and loss of service. Nonetheless the traditional prediction models are burdened with great difficulties to follow the changes in network traffic. As it can be seen as a temporal sequence, this work puts several *online deep learning* models under the microscope to elucidate whether they are able to adapt dynamically to changes in the temporal series. Various distinct architectures featuring convolutional and LSTM layers have been tested against the Milan dataset, which holds information about the traffic load in the city of Milan, Italy, between November and December 2013.

The models' outputs have been analysed via different statistical methods, demonstrating the vast superiority of the models combining convolutional and recurrent layers over the more traditional models, even when the pattern abruptly changes. The best model so far, characterized by a single CNN layer connected to an LSTM layer has been able to boost precision, measured as NRMSE, from 0.1778 to 0.142. Last but not least, all the models scrutinized by this work have been successfully optimized to achieve the greatest possible computation speeds.

Keywords

Convolutional networks, network traffic, neural networks, online learning, prediction.

Agradecimientos

A mi familia, a mis amigos, a mis profesores y, sobre todo, al Dr. D. Juan Pablo Casaseca de la Higuera. A todos ellos les agradezco su infinita paciencia.

Índice

1. Fundamentos del <i>Machine Learning</i> y <i>Deep learning</i>	4
1.1. Inteligencia Artificial	4
1.2. <i>Machine Learning</i>	4
1.2.1. Técnicas de entrenamiento	5
1.3. <i>Deep learning</i>	6
2. Redes neuronales	7
2.1. El <i>Perceptron</i>	7
2.2. Funciones de activación	9
2.3. Entrenamiento de redes neuronales y <i>backpropagation</i>	9
2.3.1. Gradient descent	9
2.3.2. Funciones de coste	10
2.3.3. <i>Backpropagation</i>	11
3. Predicción de series temporales	12

Índice de figuras

Índice de tablas

1. Fundamentos del *Machine Learning* y *Deep learning*

1.1. Inteligencia Artificial

A lo largo de la historia el ser humano ha tratado de automatizar diversas tareas para obtener una mayor eficiencia en su ejecución, reducir su complejidad o liberar al ser humano de las mismas. La programación tal y como la conocemos ahora tiene el mismo objetivo y tuvo su inicio con Ada Lovelace y su *Analytical Engine* en 1843.

El *Analytical Engine* fue el primer ordenador de uso general conocido aunque se limitaba a repetir un mismo conjunto de órdenes perfectamente definidas. En 1950, un grupo de programadores da forma a lo que hoy conocemos como Inteligencia Artificial: la automatización de tareas que requieren del intelecto humano para ser realizadas y completadas.

Un gran ejemplo de Inteligencia Artificial fue DeepBlue: un programa que, a través del procesamiento de unos datos de entrada y una serie de reglas o patrones definidos que relacionan las entradas con las salidas, era capaz de jugar al ajedrez. DeepBlue no era capaz de realizar ningún tipo de aprendizaje y, aun así, fue una Inteligencia Artificial capaz de superar a grandes maestros mundiales del ajedrez como Gary Kasparov.

Aquellas inteligencias artificiales incapaces de aprender como DeepBlue son clasificadas como inteligencias artificiales simbólicas. Sin embargo, el concepto Inteligencia Artificial es muy amplio y engloba otros conceptos como el *Machine Learning*, que engloba a su vez el *Deep learning*.

1.2. *Machine Learning*

Deep Blue fue posible gracias a que sus programadores conocían a la perfección el patrón que relacionaba las entradas, en este caso la posición actual de las figuras de ajedrez en el tablero, y las salidas, siendo éstas últimas los movimientos que DeepBlue debía realizar sobre sus figuras en su turno de juego. Este patrón estaba extremadamente bien definido en las reglas del ajedrez y DeepBlue fue programado manualmente con un conjunto muy extenso de jugadas y decisiones que cubrían todas las posibilidades de juego.

Aun con la eficiencia mostrada por DeepBlue en el ajedrez, estos ejemplos de Inteligencia Artificial resultaron ser pésimos en la ejecución de tareas más complejas y poco definidas como la clasificación de imágenes o el procesamiento de texto.

Machine Learning, también llamado aprendizaje automático, le da una vuelta al problema: en vez de calcular las salidas mediante el procesamiento de las entradas y las reglas que relacionan las entradas y las salidas, se procesan las entradas y sus correspondientes salidas para averiguar los patrones que relacionan a ambas. De

esta forma, la Inteligencia Artificial puede embarcarse en la resolución de problemas cuyas reglas no están definidas de forma precisa o son demasiado extensas o arduas de definir.

Para estimar si los patrones calculados son correctos, es necesario evaluarlos midiendo la distancia entre las salidas reales o esperadas correspondientes a ciertas entradas y las salidas obtenidas aplicando las reglas obtenidas a las entradas. La forma en la que se calcula esta distancia está definida en la función de coste o *loss function*. Esta distancia es integrada de nuevo en el algoritmo de *Machine Learning* como retroalimentación para aumentar su precisión a la hora de averiguar las reglas entre las entradas y las salidas en iteraciones sucesivas. Este paso final completa el proceso de aprendizaje o entrenamiento, que finaliza cuando la distancia entre las salidas esperadas y las salidas obtenidas es suficientemente pequeña.

Si definimos los datos de entrada y sus salidas esperadas como E y a la función de coste como R , es posible definir el *Machine Learning* o aprendizaje automático: Un programa aprende de la experiencia E con respecto a una tarea T y una métrica de rendimiento R si su rendimiento en la tarea, medido por R , mejora con más experiencia E .

1.2.1. Técnicas de entrenamiento

Existen multitud de algoritmos de que pueden ser clasificados según su método de entrenamiento en: aprendizaje por refuerzo, aprendizaje no supervisado y aprendizaje supervisado.

El aprendizaje por refuerzo utiliza un sistema de recompensas y penalizaciones para que un agente de aprendizaje automático, mediante prueba y error, ajuste sus decisiones con el fin de maximizar las recompensas obtenidas y minimizar las penalizaciones aplicadas por el sistema.

Es especialmente útil para aprender decisiones secuenciales como el guiado autónomo de un robot por una ruta definida o la optimización de unos recursos como el uso de refrigeración industrial en centros de datos. En ambos casos, el sistema de recompensas asigna una valoración positiva a aquellas decisiones que mantienen al robot en la ruta o estabilizan la temperatura de los servidores mientras evalúa negativamente aquellas decisiones que alejan desvían al robot o llevan la temperatura de los servidores fuera de un rango óptimo establecido.

Algunos ejemplos de algoritmos de aprendizaje por refuerzo son SARSA, Q-Learning...

El aprendizaje no supervisado está pensado para la organización y definición de un conjunto de datos. Los algoritmos englobados en esta técnica de entrenamiento deben encontrar los patrones y estructuras subyacentes en los datos de entrada, sin esperar una salida relacionada con los mismos. El aprendizaje no supervisado busca agrupar los datos en distintos sectores que maximicen la distancia intersector y minimicen la distancia intrasector.

Es especialmente útil en la clasificación de los datos según sus características como la segmentación de clientes, en la detección de anomalías como los fraudes financieros o en la reducción de la dimensionalidad de los datos como la simplificación de bases de datos. Algunos ejemplos de algoritmos de aprendizaje no supervisado son *k-means clustering*, *Support Vector Clustering* o el análisis de componentes principales (PCA).

Finalmente, el aprendizaje supervisado, en donde se enclaustra este trabajo, utiliza pares entrada-salida perfectamente definidos para entrenar sus algoritmos. Estos pares son conocidos como datos etiquetados y tienen la función de actuar como referencia para la medición de la distancia entre la salida obtenida y la salida esperada. Los modelos de aprendizaje supervisado deben, utilizando dichas distancias, ajustar de la forma más precisa posible el patrón que relaciona las entradas y las salidas.

Es especialmente útil en la clasificación como, por ejemplo, la identificación de correos electrónicos de SPAM y la predicción de datos como la predicción de eventos meteorológicos. Algunos ejemplos de algoritmos de aprendizaje supervisado son los árboles de decisión, *Support Vector Machines*, la regresión lineal y no lineal y las neuronas artificiales.

1.3. *Deep learning*

Con todo esto, los algoritmos de *Machine Learning* son capaces de resolver problemas más complejos que la Inteligencia Artificial simbólica pero es posible ir un paso más allá. Aunque el *Machine Learning* pueda aprender los patrones que relacionan unas entradas con unas salidas, el *Deep learning*, también llamado aprendizaje profundo, utiliza varias capas jerárquicas de aprendizaje para combinar las estructuras más simples obtenidas en las primeras capas en patrones más complejos que representen los datos procesados de forma mucho más precisa.

En *Deep Learning*, los algoritmos utilizan capas de neuronas artificiales organizadas de forma sucesiva. Existe una gran variedad de neuronas artificiales, cada una de ellas especializada en una tarea concreta como aplicar un mismo filtro a un conjunto de datos de entrada, almacenar información relevante sobre los datos o reestructurar y redimensionar los datos procesados.

Al tener múltiples capas de neuronas, los modelos de aprendizaje profundo son capaces de procesar los datos sin ninguna intervención humana, mientras que los algoritmos de *Machine Learning* necesitan de la intervención del ser humano para refinar y adaptar los datos manualmente a las necesidades de cada algoritmo. Sin embargo, los modelos de *Deep Learning* necesitan grandes cantidades de datos para poder ser entrenados, lo que dificulta enormemente la interpretación de los procesos llevados a cabo por las redes neuronales, además de una gran inversión en componentes electrónicos especializados en dichos cálculos como las GPUs (graphical Processing Units). Un ejemplo de esta inversión se encuentra en el reciente giro radical de la producción de GPUs de Nvidia, el mayor fabricante de GPUs a nivel mundial, con el nuevo objetivo de satisfacer las ingentes necesidades del mercado del *Deep Learning*.

Aunque las redes neuronales nacieron en 1943 con el modelo lógico de neurona artificial de McCulloch y Pitts, no fueron rentables hasta el desarrollo de nuevas arquitecturas y funciones de activación, métodos de aprendizaje y ajuste como *back-propagation* y la mejora de los componentes electrónicos disponibles. En la actualidad, los modelos de aprendizaje profundo son los algoritmos predeterminados y más eficientes en prácticamente todo el área de Inteligencia Artificial, desde visión artificial hasta procesamiento de lenguaje.

En estos últimos años desde la expansión del aprendizaje profundo, se han conseguido hitos como, entre otros, la clasificación de imágenes o el procesamiento de audio y lenguaje a nivel humano, la aparición de asistentes digitales y modelos conversacionales como ChatGPT o Microsoft Copilot o la mejora de sistemas de recomendaciones y búsquedas web.

Este trabajo se enmarca dentro del ámbito del *Deep Learning*, más concretamente en la predicción de tráfico de red a través de redes neuronales profundas. La predicción de series temporales, como es el caso del tráfico de red, es una tarea ardua que conlleva la estimación del patrón existente en la serie. Por lo tanto, cumple con todos los requisitos mencionados anteriormente para ser automatizada mediante algoritmos de aprendizaje profundo.

2. Redes neuronales

Las redes neuronales son los modelos de aprendizaje profundo por excelencia. Están formadas por neuronas artificiales que utilizan una serie de pesos para procesar unos datos de entrada. Su primera aparición fue en 1943 como una simple descripción del funcionamiento de las neuronas presentes en el sistema nervioso animal: una neurona con salida binaria activará su salida cuando un cierto número de sus entradas, también binarias, estén activas.

2.1. El *Perceptron*

Aunque las neuronas binarias pueden combinarse para formar redes capaces de procesar expresiones lógicas complejas, no son más que una combinación de puertas lógicas como las presentes en cualquier circuito electrónico. En 1957 Rosenblatt redefinió las neuronas artificiales para eliminar sus restricciones binarias, permitiendo un diseño basado en entradas y salidas continuas. Estas neuronas son llamadas TLU (*Threshold Linear Unit*).

Las TLUs reciben como entradas una serie de valores x_j donde J es la cantidad de entradas recibidas. Las TLUs calculan la combinación lineal z de las entradas con una serie de pesos w_j asociados a cada entrada y un término independiente b . Seguidamente, se aplica una función de activación $\phi(z)$ que acota la salida y de las TLUs en un rango de valores típicamente comprendido entre -1 y 1 o entre 0 y 1. De esta forma se evita que la salida de las TLUs crezca hasta infinito si se encadenan

varias TLUs.

$$z = b + \sum_{j=1}^J w_j * x_j = \mathbf{w}^T \mathbf{x} + b \quad (1)$$

$$y = \phi(z) \quad (2)$$

Una neurona TLU con una función de activación binaria como la Heaviside puede ser utilizada en tareas como la clasificación binaria. Sin embargo, su verdadero poder reside en la agrupación de varias TLUs en una misma capa. Esta es la arquitectura de redes neuronales más básica: el *Perceptron*, capaz de, por ejemplo, clasificar los datos en $J+1$ categorías siendo J el número de neuronas TLU presentes en la arquitectura.

Cada TLU está conectada a todas las entradas en una disposición llamada capa completamente conectada o *dense layer*. Esta disposición no está presente en todas las arquitecturas como, por ejemplo, en las neuronas convolucionales. La Figura ?? describe un *Perceptron* de dos entradas y tres TLUs con tres salidas. En este caso, en forma matricial con tantas filas i como TLUs y tantas columnas j como entradas, los pesos $w_{i,j}$ se agrupan en la matriz $\mathbf{W}$ y las entradas y los términos independientes en los vectores $\mathbf{x}$ y $\mathbf{b}$ respectivamente, modificando las expresiones 1 y 2:

$$\mathbf{z} = \mathbf{W}^T \mathbf{x} + \mathbf{b} \quad (3)$$

$$\mathbf{y} = \phi(\mathbf{z}) \quad (4)$$

Los *Perceptrons* son en sí arquitecturas muy simples con capacidades limitadas pero pueden encadenarse para aumentar su capacidad de cómputo. Esta arquitectura se llama *Perceptron* multicapa o MLP (*Multilayer Perceptron*). Así, las salidas de la primera capa de neuronas TLU es utilizada como entrada para la segunda capa de TLUs de forma sucesiva, como se observa en la Figura ??.

Denotando como $\mathbf{h}_l$ la salida de la capa l en un MLP con L capas y a los pesos de las neuronas de la misma capa como $\mathbf{W}_l$ y $\mathbf{b}_l$, se modifican las ecuaciones 3 y 4 de la forma:

$$\mathbf{h}_l = \phi(\mathbf{W}_l^T \mathbf{h}_{l-1} + \mathbf{b}_l) \quad (5)$$

$$\mathbf{y} = \mathbf{h}_L \quad (6)$$

Los MLPs pueden ser utilizados en diversas tareas como la clasificación o la regresión. Son capaces incluso de predecir varios valores al mismo tiempo dependiendo del número de TLUs presentes en la última capa de neuronas.

2.2. Funciones de activación

Existe una gran multitud de funciones de activación, todas ellas no lineales, para cada tarea en la que se apliquen las neuronas artificiales. Su implementación permite que las neuronas realicen transformaciones no lineales de los datos, siendo así capaces de ajustarse a una mayor variedad de patrones. Sin embargo, las funciones más básicas como la Heaviside no son compatibles con los métodos actuales de entrenamiento de modelos de aprendizaje profundo. Por ello, se propusieron otras funciones más complejas como la función sigmoide $\sigma(z)$, la función linear rectificada $ReLU(z)$ (*Rectified Linear Unit*) y la tangente hiperbólica $\tanh(z)$.

Las funciones de activación han sufrido un empujón en los últimos años para eliminar las áreas de las funciones como ReLU con gradientes nulos o puntos no derivables. Algunas de ellas son Leaky ReLU y Mish.

2.3. Entrenamiento de redes neuronales y *backpropagation*

Como se ha mencionado anteriormente, además de neuronas y funciones de activación, es necesario dotar al modelo de aprendizaje profundo de una métrica que evalúe la precisión de sus resultados y un método matemático capaz de modificar la red neuronal basándose en dicha métrica. Para obtener salidas distintas, la forma más sencilla es modificar ligeramente los valores contenidos en la matriz de pesos $\mathbf{W}$ de forma aleatoria. Sin embargo, existen protocolos más eficaces como *gradient descent*.

2.3.1. Gradient descent

Gradient descent es un algoritmo de entrenamiento que modifica los parámetros que influyen en la salida de un modelo con el objetivo de minimizar una función de coste. Los parámetros de una red neuronal son los pesos $\mathbf{W}$ incluyendo los términos independientes $\mathbf{B}$. La variación introducida en los parámetros está controlada por la tasa de aprendizaje o *learning rate* η .

Con cada cambio en los parámetros, es posible calcular cuánto varía la función de coste a través de su gradiente con respecto a los parámetros modificados $\nabla_{\mathbf{w}}$. Un gradiente positivo indica la dirección en la que la función de coste aumenta, por lo que los cambios deberán ir en la dirección opuesta. De esta forma, denominando LF a la función de coste utilizada, el algoritmo *gradient descent* se resume en:

$$\mathbf{W}^t = \mathbf{W}^{t-1} - \eta \nabla_{\mathbf{w}} LF(\mathbf{W}^{t-1}) \quad (7)$$

donde $\mathbf{W}^{t-1}$ son los parámetros iniciales y $\mathbf{W}^t$ son los parámetros resultantes.

A lo largo de múltiples iteraciones, *gradient descent* es capaz de obtener los pesos adecuados para cada neurona de tal forma que minimicen la función de coste apli-

cada. Sin embargo, las funciones de coste presentan valles que podrían confundir al algoritmo *gradient descent* haciéndole creer que se encuentra en el mínimo absoluto. Por ello, es común añadir un término aleatorio α que permita escapar de los mínimos locales. Añadiéndolo a la Ecuación 7 se consigue la expresión para *stochastic gradient descent*:

$$\mathbf{W}^t = \mathbf{W}^{t-1} - \eta\alpha\nabla_{\mathbf{W}}LF(\mathbf{W}^{t-1}) \quad (8)$$

Estas variaciones en la expresión matemática que define el algoritmo *gradient descent* son los llamados optimizadores. Existen una gran cantidad de ellos como SDG (*Stochastic Gradient Descent*), *Momentum*, RMSProp o Adam. Cada uno de ellos implementa una mejora de la eficiencia respecto al anterior.

En este trabajo se utiliza un optimizador para aumentar la eficiencia de *gradient descent* propuesto en [?]. Partiendo de la premisa de que las iteraciones de *gradient descent* que producen un error mayor en la función de coste ofrecen más información que aquellas con errores más pequeños, dar un peso mayor a las primeras aumentará la eficacia del algoritmo. Para ello se modifica la Ecuación 7 añadiendo la norma euclídea al cuadrado del error cometido en cada iteración:

$$\mathbf{W}^t = \mathbf{W}^{t-1} - \eta \left\| \frac{y - \hat{y}_{t-1}}{y} \right\|^2 \nabla_{\mathbf{W}}LF(\mathbf{W}^{t-1}) \quad (9)$$

donde y es la salida real esperada e $\hat{y}_{t-1}$ es la salida arrojada por la red neuronal en la iteración $t - 1$ de *gradient descent*.

2.3.2. Funciones de coste

Las funciones de coste son una parte esencial del algoritmo *gradient descent* y no están relacionadas con las funciones de activación. Son el objetivo a minimizar a través de la modificación de los parámetros de la red neuronal. El cambio en los parámetros modifica el procesamiento que realiza la red sobre los datos de entrada y, consecuentemente, en la salida obtenida. La función de coste debe ajustarse al objetivo de la red neuronal para que los resultados sean válidos.

El algoritmo *gradient descent* permite a la red neuronal tomar cualquier atajo para minimizar su función de coste como ignorar ciertos datos clave o inventar nuevos datos. Por ello, ante la gran diversidad de funciones de coste existente a día de hoy, no todas las funciones son implementables para todas las redes neuronales ni para todos los propósitos.

Entre las funciones de coste más populares se encuentran: la entropía cruzada binaria para tareas de clasificación binaria, su variante categórica para clasificación multiclase, similitud del coseno para procesamiento de textos o el error cuadrático medio (MSE) para regresión y procesamiento de series temporales. En este trabajo se utilizará la función de coste MSE (*Mean Squared Error*).

2.3.3. *Backpropagation*

El algoritmo *gradient descent* es la base del entrenamiento de redes neuronales pero no es aplicable directamente a redes con múltiples capas de neuronas. En 1970 Seppo Linnainmaa introdujo el algoritmo de diferenciación automática invertida (*reverse-mode automatic differentiation*) que calcula en dos pasadas a toda la red todos los gradientes del error cometido en la salida, medido por la función de coste, respecto a todos y cada uno de los parámetros de la red neuronal. De esta forma se obtiene la contribución al error de cada término independiente $b_{i,l}$ y cada peso $w_{i,j,l}$ de cada entrada j de cada neurona i de cada capa l . El algoritmo *gradient descent* puede utilizar estas aportaciones al error para calcular la magnitud en la que cada parámetro debe ser modificado.

El algoritmo *backpropagation*, también llamado retropropagación, es la combinación del algoritmo de Linnainmaa con *gradient descent*. En primera instancia, la red neuronal procesa los datos de entrada para calcular una salida. Este es el pase hacia delante o *forward pass*. Es el funcionamiento normal de una red neuronal salvo que se guardan todas las salidas de todas las neuronas de todas las celdas en vez de desecharlas y dar únicamente la salida final. Con todas esas salidas se ejecuta *reverse-mode automatic differentiation* desde la última capa hasta la primera para calcular la contribución al error perteneciente a cada capa. Este es el pase hacia atrás o *backward pass*. Por último, todos los parámetros son modificados con *gradient descent*.

Es muy importante que los parámetros estén inicializados de forma aleatoria para que se puedan distinguir los unos de los otros. Si no fuera así, el algoritmo *reverse-mode automatic differentiation* sería incapaz de saber qué neurona ha contribuido más o menos al error y *gradient descent* las actualizaría en la misma cuantía. Al ser todas las neuronas iguales, actuarían como una sola con la consecuente reducción de la capacidad de cómputo de la red neuronal.

Por motivos similares no todas las funciones de activación son válidas. Aunque la función de coste indica el error final cometido en la salida, *backpropagation* debe calcular la contribución de cada parámetro al total. Cambiar un peso $w_{i,j,l}$ o un término independiente cualquiera $b_{i,l}$ de cualquier neurona no supone un cambio directo en la salida de dicha neurona sino que ha de pasar por la función de activación. Por lo tanto, la función de activación debe ser diferenciable para que *backpropagation* pueda calcular cuánto ha de modificar el peso o término término independiente deseado para mover la salida final en una dirección determinada.

Por esta razón, todas las funciones no diferenciables utilizan trucos para sustituir sus puntos no diferenciables con otros valores. Sin embargo, las funciones de activación más problemáticas son aquellas con gradientes nulos puesto que anulan el efecto de *gradient descent*. Un gran ejemplo de gradientes nulos es la función Heaviside(z), que tiene gradiente cero para todo z salvo $z = 0$ donde no es diferenciable.

La función de activación utilizada en este trabajo y en la gran mayoría de redes neuronales es ReLU(z). Tiene un gradiente no nulo para $z > 0$, un punto no diferenciable en $z = 0$ y gradiente nulo para $z < 0$. Esto implica que, si la combinación

de entradas y pesos de la neurona es menor a cero, la neurona queda inutilizada. Es una gran ventaja que permite a la red neuronal eliminar neuronas innecesarias para cada dato de entrada.

3. Predicción de series temporales